Corvinus University of Budapest

The Effect of Large Language Model Training Cutoffs on the Diffusion of New Software Libraries

Thesis Proposal

Author: Roland Tuboly

roland.tuboly@stud.uni-corvinus.hu

Supervisor: Johannes Wachs

johannes.wachs@uni-corvinus.hu

2025.

Motivation and Research Question

The integration of Large Language Models (LLMs) into the software development lifecycle represents a big shift in how code is produced, debugged, and discovered. By automating routine syntax generation and reducing the friction of information retrieval, generative AI offers substantial productivity gains. However, unlike human developers who continuously update their knowledge base through real-time exposure to the ecosystem, pre-trained LLMs operate under a unique and rigid technical constraint: they are bounded by specific knowledge cutoff dates. An LLM's knowledge is limited strictly to the libraries, frameworks, and syntax patterns that existed prior to the end of its training data collection. This thesis will investigate whether this constraint creates a disadvantage for libraries released shortly after major LLM training cutoffs, thereby introducing a systemic bias into the diffusion of innovation within the Python environment.

The core hypothesis of this research is that LLMs act as powerful intermediaries in the market for software tools. Before the widespread of this technology, the diffusion of a new library was driven by peer recommendation, search engine optimization (SEO), and community visibility on platforms like Reddit. In the current era, developers increasingly rely on AI coding assistants (such as GitHub Copilot or Claude Code) to suggest imports and solve problems. If a library is included in the model's training data (meaning it has been released prior to the cutoff) the model can effectively recommend it as a solution, explain its documentation, and put out syntactically correct code that utilize it. Conversely, libraries released shortly after the cutoff are effectively invisible to the model's internal logic. When a developer queries the model for a solution that the new library addresses, the model will recommend an older, known alternative, which gives big advantage to incumbent technologies and raising the search costs for novel solutions.

This dynamic reflects a profound tension between individual enlargement and collective narrowing. Recent experimental evidence by Doshi and Hauser (2024) in the domain of creative writing demonstrates that while access to generative AI enhances the quality of individual output, it simultaneously reduces the collective variance and novelty of the content produced. They characterize this as a social dilemma: writers are individually better off using the tool, but the aggregate ecosystem suffers from a narrower scope of idea. Applied to the context of mainly software engineering, this suggests a similar risk: while LLMs may help

individual developers write code faster by recalling established patterns, they may miss out on superior but “unknown” libraries (Noy & Zhang, 2023). If this hypothesis holds, the widespread adoption of AI coding tools could lead to a solidifying of the software stack, where the "winners" are determined by their presence in the training data of dominant large language models.

Therefore, the primary research question of this thesis is: Do Python libraries released shortly before a major LLM training cutoff diffuse more widely than those released shortly after? To answer this, I will estimate the causal effect of being "known" by dominant LLMs on two distinct dimensions of diffusion: aggregate market interest, measured by download counts from the Python Package Index (PyPI), and actual implementation, measured by usage frequency in GitHub repositories. I will leverage this and another dataset collected by Professor Johannes Wachs, which distinguishes between code written by humans (who can search the live web) and code generated by artificial intelligence (which are constrained by their training cutoffs).

Data

The analysis will rely on a comprehensive panel dataset constructed by merging three primary sources: library metadata from PyPI, aggregate download statistics, and code usage data from GitHub.

The primary measure of broad market diffusion will be aggregate download counts from the Python Package Index, the official third-party software repository for the Python programming language (pypi.org, 2025). For each library in the sample, there are weekly download counts, ensuring a high-frequency view of adoption trends. A critical component of this data is the release timestamp for every version of a library (I hope this is available in the collected data). The analysis (if possible) will focus on the initial release date of new libraries ("born" packages) rather than updates to existing ones. This distinction would be vital because established libraries have pre-existing user bases and adoption trends that would confound the analysis. By focusing on libraries that start existing within a symmetric bandwidth around major LLM cutoff dates, I could isolate the effect of the cutoff on the initial trajectory of diffusion. The dataset will be cleaned (if needed) to remove spam packages and toy projects, filtering for libraries that meet a minimum threshold of viability (e.g., non-trivial code size, presence of documentation) to ensure the analysis captures meaningful libraries only.

To capture the specific mechanism of AI assistance - and to verify that diffusion is being driven by actual usage rather than automated mirrors - I will utilize a commit-level dataset from GitHub. This tracks the presence of library imports in public code repositories over time. Just presence, however, is insufficient to prove the above stated hypothesis. To distinguish whether diffusion is driven by AI generation or general developer adoption, I will utilize the commit-level AI Score too.

Following the rigorous methodology detailed by Wachs et al. (2025), I will utilize this data that was created by the classification system based on GraphCodeBERT, a pre-trained language model for code that incorporates semantic structure. This classifier is trained on a ground-truth dataset of human-written code and synthetic AI-generated code (produced by models like GPT-4) to detect the subtle stylistic fingerprints of generative AI. With a reported AUC[1] of roughly 0.96, this method provides a reliable probability score for each commit. I will aggregate these scores to the library-week level to construct two parallel time series for each library: human and AI-driven usage. This decomposition is central to the thesis, as it allows for a direct test of the mechanism: if the hypothesis is correct, the discontinuity at the cutoff should be driven largely by the AI-driven usage series.

The treatment in this regression discontinuity design is determined purely by the library's release date relative to the known training cutoffs of dominant models. The analysis will primarily focus on the September 2021 cutoff, which is associated with the training data of GPT-3.5 and GPT-4 (platform.openai.com, 2025; learn.microsoft.com, 2025) and the initial versions of GitHub Copilot that drove mass adoption. A library is considered treated if its initial release date is strictly prior to the cutoff date. It is untreated if it is after the cutoff. While newer models have later cutoffs, e.g., GPT-4o in late 2023 (learn.microsoft.com, 2025), the 2021 cutoff represents the cleanest natural experiment due to the shock of ChatGPT's release in late 2022 (openai.com, 2025), where the model suddenly became a primary search interface for millions of developers while relying on data that was over a year old. The final dataset will be structured as a balanced panel of libraries observed over the first 52 weeks of their existence, with treatment status fixed by their initial release timestamp.

[1] Area under the curve

Empirical Strategy

The primary identification strategy is a sharp regression discontinuity design (RDD). The intuition underlying this design is that libraries released immediately before and immediately after the cutoff date are comparable in terms of unobserved quality, author ambition, and potential for adoption. In the absence of the cutoff date, we would expect the adoption of libraries to evolve consistently over time. Any jump in adoption for libraries released just before the cutoff can therefore be attributed to their inclusion in the LLM's training set (Angrist & Pischke, 2009).

I will estimate the local average treatment effect (LATE) of being in the training set on subsequent diffusion. For a library i, let Y_i represent the cumulative downloads or usage at a fixed time horizon, such as e.g. six months post-release (I think I would analyze different time windows so I can compare these as well). The core RDD specification is given by:

$$Y_i = \alpha + \tau D_i + f(R_i - C) + \epsilon_i$$

Where Y_i is the (logged) outcome of interest, R_i is the library's release date (the running variable), and C is the LLM knowledge cutoff date. The term $D_i = 1\ (R_i \leq C)$ is the treatment indicator. The function $f(R_i - C)$ is a flexible polynomial of the running variable, allowed to have different slopes on either side of the cutoff. I will estimate this using local linear regression with a triangular kernel, which has been shown to be optimal for boundary estimation (Cheng, Fan, & Marron, 1997) and is the standard approach in the RDD literature (Imbens & Lemieux, 2008; Calonico, Cattaneo, & Titiunik, 2014). The coefficient of interest is τ, which measures the jump in diffusion at the cutoff. A positive, statistically significant τ would indicate that being "known" to the AI significantly boosts a library's adoption.

Validity

The validity of this design rests on the continuity assumption: that the potential adoption of libraries would evolve consistently through the cutoff date in the absence of the treatment. There are two primary threats to that could collide with this assumption. First is manipulation of the running variable: if library authors knew the cutoff date in advance and rushed to release their packages to ensure inclusion, estimates would be biased. However, for the September 2021 cutoff, this is implausible; the exact data collection windows of OpenAI's models were not public knowledge at the time, and the mass adoption was significantly later (late 2022). Nevertheless, I will formally test this by checking for a discontinuity in the density of library releases at the cutoff with a McCrary density test (McCrary, 2008).

The second threat is coinciding shocks. If the cutoff date happens to align perfectly with another major event, the effect might be confounded. To mitigate this, I will check the release dates of different Python versions (e.g., Python 3.10) to ensure they do not overlap with the treatment window.

Robustness Checks

To ensure the stability of the results, I will perform a suite of robust checks.

I will re-estimate the model using a range of bandwidths (e.g. 2, 4, 8, 12 weeks) to ensure the result is not driven by an arbitrary choice of window size.

Since the exact "cutoff" might be a fuzzy window of a few days or weeks where data collection tapered off, I will estimate a donut RDD that excludes observations in the immediate 2-week vicinity of the cutoff (Barreca et. al, 2011). This removes libraries whose treatment status is ambiguous.

I will run the entire analysis on fake cutoff dates (e.g., September 2020 or September 2022) where no LLM training discontinuity exists. Significant effects at these dates would indicate model misspecification or seasonal trends (Ludwig & Miller, 2007).

I will test robustness to quadratic polynomials and uniform kernels to ensure the results are not artifacts of the local linear specification.

References

PyPI, Python Package Index (2025). https://pypi.org

Noy, S., & Zhang, W. (2023). Experimental evidence on the productivity effects of generative artificial intelligence. Science, 381(6654), 187-192.

Doshi, A. R., & Hauser, O. P. (2024). Generative AI enhances individual creativity but reduces the collective diversity of novel content. Science advances, 10(28), eadn5290.

Daniotti, S., Wachs, J., Feng, X., & Neffke, F. (2025). Who is using AI to code? Global diffusion and impact of generative AI. arXiv preprint arXiv:2506.08945.

OpenAI Platform (2025). https://platform.openai.com/docs/models/gpt-3.5-turbo

Microsoft Learn (2025). https://tinyurl.com/9zaktsyk

OpenAI (2025). https://openai.com/index/chatgpt/

Angrist, J. D., & Pischke, J. S. (2009). Mostly harmless econometrics: An empiricist's companion. Princeton university press.

Calonico, S., Cattaneo, M. D., & Titiunik, R. (2014). Robust nonparametric confidence intervals for regression-discontinuity designs. Econometrica, 82(6), 2295-2326.

Imbens, G. W., & Lemieux, T. (2008). Regression discontinuity designs: A guide to practice. Journal of econometrics, 142(2), 615-635.

Cheng, M. Y., Fan, J., & Marron, J. S. (1997). On automatic boundary corrections. The Annals of Statistics, 25(4), 1691-1708.

McCrary, J. (2008). Manipulation of the running variable in the regression discontinuity design: A density test. Journal of econometrics, 142(2), 698-714.

Barreca, A. I., Guldi, M., Lindo, J. M., & Waddell, G. R. (2011). Saving babies? Revisiting the effect of very low birth weight classification. The quarterly journal of economics, 126(4), 2117-2123.

Ludwig, J., & Miller, D. L. (2007). Does Head Start improve children's life chances? Evidence from a regression discontinuity design. The Quarterly journal of economics, 122(1), 159-208.

Statement on the Use of Artificial Intelligence

This research proposal utilized Large Language Models (specifically OpenAI's ChatGPT and Google's Gemini) as a productivity tool and technical collaborator. In accordance with academic integrity guidelines, I served as the primary decisionmaker, reviewing all AI-generated suggestions for theoretical soundness. Draft text sections were iteratively refined using AI to ensure precise academic terminology and tonal consistency. The LLMs assisted in synthesizing the literature review to better connect the cited studies to each other. While AI tools assisted me, I retain full responsibility for the final content, the accuracy of the citations and the scientific conclusions presented in this proposal.